

Uke 5 – Grunnleggende SQL

Spørringer

- SQL-spørringer har formen:
SELECT <attributter / kolonner> – alle attributer som skal vises i svaret
FROM <tabeller>
WHERE <betingelser> – velger(selekterer) alt som oppfyller
kriteriet/betingelsene
- Relasjonsalgebra:
- Projeksjon (π) = SELECT
- Seleksjon (σ) = WHERE-betingelser
- $\pi_{\text{select}}(\sigma_{\text{where}}(\text{tabeller}))$

SELECT

Vi har tabellen: film(filmid, title, prodyear)

- SELECT filmid FROM film
gir oss bare filmid
- SELECT filmid, title FROM film
gir oss filmid og title
- SELECT * FROM film
gir oss alle attributtene/kolonnene
- SELECT DISTINCT
fjerner duplikater

Mer SELECT

Vi har tabellen: Person(personid, firstname, lastname, gender)

- `SELECT concat(firstname, ' ', lastname) AS Name FROM Person`

Eller:

- `SELECT firstname || ' ' || lastname AS Name FROM Person`

Slår sammen 2 kolonner til 1

- Kan også bruke matematiske operasjoner som +, -, * og / rett i select-delen.
- F.eks: `SELECT 2*2+4-1;` Vil gi tallet 7.

SELECT med aggregering

Kan også bruke aggregering i select-delen. De forskjellige er:

- sum – summen
- avg – gjennomsnitt
- max – maksimum
- min – minimum
- count – teller alle rader (ikke null verdier)
- count(*) teller alle rader inkludert null verdier

Eksempler (filmid, title, prodyear):

SELECT count(filmid) – gir antall filmid i film-tabellen

SELECT max(prodyear) – gir deg den/de nyeste filmene i databasen (høyest tall)

Where-klausulen

Hvis vi har tabellen:

film(filmid, title, prodyear), og skal velge ut navn på alle filmene som er produsert i perioden 2006-2008, så bruker vi where-betingelsen til å filtrere ut filmene i den perioden.

F.eks slik:

```
SELECT title FROM film  
WHERE prodyear > 2005 AND prodyear < 2009;
```

Eller med disse WHERE-betingelsene:

```
WHERE prodyear >= 2006 AND prodyear <= 2008  
WHERE prodyear BETWEEN 2006 AND 2008
```

Kan kombinere flere WHERE-betingelser med "AND" , "OR" , "NOT"

Søke i tekst

Til det bruker man LIKE og %.

Hvis vi ser på tabellen film(filmid, title, prodyear) igjen, så vil:

```
SELECT title FROM film
```

```
WHERE title LIKE 'Lord of the Rings';
```

 vil gi oss akkurat filmen(e) med den tittelen.

- '%Lord of the Rings', tittelen slutter på "Lord of the Rings"
- 'Lord of the Rings%', tittelen starter på "Lord of the Rings"
- '%Lord of the Rings%', kommer noe før og etter 'Lord of the Rings'

Viktig:

På tekststrenger er det veldig viktig å skrive nøyaktig det man blir bedt om. Den er sensitiv på store forbokstaver, mellomrom, tegn etc.

f.eks:

'Lord of the Rings%' og 'Lord of the Rings %' vil ikke gi det samme svaret. På sistnevnte kommer det et mellomrom etter Rings før det kommer noe mer, så filmen med navnet "Lord of the Rings: The Two Towers" ville ikke blitt med siden det kommer et ":" etter Rings

NULL-verdier

NULL-verdier er ukjente verdier. Kan ikke bruke "=" eller "LIKE" for å finne disse. Man bruker:

- IS NULL og IS NOT NULL

F.eks:

```
SELECT * from film
WHERE prodyear IS NULL
LIMIT 3;
```

filmid	title	prodyear
3562	'Girls of Phoenix' 140, The	
11306	...hogy magának milyen mosolya van!	
11634	Camiling Story	

(3 rows)

Joins i SQL

For å slå sammen tabeller i dette kurset bruker vi disse forskjellige metodene:

- INNER JOIN
- NATURAL JOIN
- SELF JOIN (Side 67-73 i foiler om joins, 21. september)

og disse som blir introdusert senere i kurset:

- FULL OUTER JOIN
- RIGHT OUTER JOIN
- LEFT OUTER JOIN

INNER JOINS

I filmdatabasen har film-tabellen en fremmednøkkel som er slik:
film(filmid) --> filmitem(filmid)

Med INNER JOIN så joiner man på en condition.

```
SELECT * from film f  
INNER JOIN filmitem fi ON f.filmid = fi.filmid  
WHERE prodyear = 2011;
```

NATURAL JOIN

Joiner på alle kolonner med likt navn.

film(filmid) --> filmitem(filmid),

Her ser vi at filmid har samme navn i begge tabellene, så her kunne vi brukt natural join. Da ser spørringen fra forrige slide slik ut:

```
SELECT * FROM film f
NATURAL JOIN filmitem fi
WHERE prodyear = 2011;
```

Men hvis vi ser på tabellen series i filmdatabasen, så har den en fremmednøkkel som er slik:

series(seriesid) --> filmitem(filmid).

Her ser vi at seriesid ikke er likt som filmid, så her funker ikke natural join, men inner join funker!

NATURAL JOIN

- I tillegg projiserer den vekk de dupliserte kolonnene
- Trenger derfor aldri gi tabellene navn (i resultatet av en naturlig join vil det aldri finnes kolonner med likt navn)
- Merk: Må være sikker på at vi ønsker å joine på ALLE kolonnene med likt navn!

Nestede spørringer

- Husk at tingene i en FROM-klausul er tabeller
- Husk også at resultatet av en SELECT-spørring er en tabell
- Så, vi kan putte en SELECT-spørring i FROM-klausulen som en tabell!

```
SELECT <kolonner>  
FROM (  
    SELECT <kolonner>  
    FROM <tabell>  
    WHERE <betingelse>  
    ) AS tab  
WHERE <betingelse>
```

Eks:

For å finne antall unike kombinasjoner av land og by for alle kunder:

```
SELECT count(*)  
FROM (SELECT DISTINCT country , city FROM customers ) AS d
```

Eks:

- Finne navnet på alle produkter med en “supplier” fra Tyskland:

```
SELECT product_name
```

```
FROM products
```

```
WHERE supplier_id IN (SELECT supplier_id FROM suppliers
```

```
WHERE country = 'Germany ')
```